

Hybrid MPI+OpenMP Performance of PETSc on ARCHER2

Leyan li

Date

MSc in High Performance Computing (with Data Science)
The University of Edinburgh
2026

Abstract

This is the bit where you summarise what is in your thesis. A non-expert reader should be able to understand the area of your work, what the main challenges were and what contributions or insights you've achieved, and gauge whether this work is relevant to their interests,. Think of the abstract as a super-concentrated summary of your MSc project; advertise your work but try to avoid cliffhangers.

A good abstract fits to 1 to 2 paragraphs and takes around 200 words to get your point across. To give you an idea, this abstract is 97 words long.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims and Research Questions	1
1.3	Contributions	1
1.4	Dissertation Structure	1
2	Background and Related Work	2
2.1	Parallel Programming Models	2
2.2	Limitations of Flat MPI	2
2.3	Hybrid MPI+OpenMP	3
2.4	ARCHER2 Node Architecture and NUMA	3
2.5	PETSc and Krylov Solvers	4
2.6	Performance Analysis Tools	5
2.7	Summary and Research Gap	5
2.8	Dissertation Report	5
3	Experimental Platform and Methodology	6
3.1	Hardware and Software Environment	6
3.2	Benchmark Problems and Drivers	7
3.3	Solver Configuration	7
3.4	Configuration Space and Selection Rules	9
3.5	Job Configuration and Thread Placement	10
4	2D and 3D Problems	11
4.1	Experimental Setup	11
4.1.1	Weak-Scaling Chains	12
4.1.2	Strong-Scaling Design	12
4.2	Weak Scaling at Constant Work per Core	12
4.2.1	2D	13
4.2.2	3D	14
4.2.3	Layout Ranking and Efficiency	16

4.3	Strong Scaling at Fixed Problem Size	17
4.3.1	2D	17
4.3.2	3D	19
4.3.3	The 32-Node Endpoint	21
4.4	Interpretation	22
5	Profiling and Bottleneck Analysis	24
5.1	Why a Repeated-Solve Driver	24
5.2	Analysis Methodology	24
5.3	MAP Profiling Results	25
5.4	Bottleneck Identification	28
5.4.1	What is Measured	29
5.4.2	Halo Exchange Dominates the Exposure, and Hybridisation Reduces It	29
5.4.3	Reduced share Exposure	31
5.4.4	What This Does and Does Not Establish	35
6	Discussion	37
6.1	RQ1: Does the threaded backend reduce time to solution, and by how much?	37
6.2	RQ2: DAt which balance, and how does the answer change with the regime?	38
6.3	RQ3: Which parts of the solve account for the difference?	38
6.4	RQ4: How far do the trends generalise, and what would establish a mechanism?	38
7	Conclusion and Future Work	39
A	Supplementary Figures	40
A.1	Weak-scaling throughput	40
A.2	Strong-scaling runtime grids	42
B	Stuff which is too detailed	43
C	Stuff which no-one will read	44

List of Tables

2.1	The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation; it inherits both cost columns.	2
2.2	The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores.	4
2.3	Operations in one GAMG-preconditioned CG step, with the PETSc event that records each and the number of calls per step measured in a representative run. A step is one preconditioner application.	5
3.1	Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging. . .	8
4.1	Grids and node counts of the six scales. The strong-scaling column is not limited by the unknowns-per-core filter of Section 3.4; its 32-node point sits deliberately below the window.	11
4.2	The 11 admitted size–node pairs, ordered into two weak-scaling chains.	12
4.3	Fastest full-node layout at every admitted size–node pair, by median throughput. A margin of 0.0% means flat MPI was fastest.	16
4.4	Weak-scaling efficiency (%) at the largest node count of each chain, against each layout's own one-node time.	17
4.5	Fastest full-node layout at each node count for the fixed 20M problems, with its advantage over flat MPI.	21
4.6	The 32-node endpoint of the strong-scaling curves. Only time per solve compares layouts directly.	22
4.7	The two dimensions on the quantities this chapter compares them by. .	23
5.1	Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8 .	25

5.2	Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. “OpenMP” is the GNU runtime’s spin and wait paths, that is threads with no work to do. Rows sum to under 100% because libc, the loader and the sampler make up a small remainder that is omitted. MAP instrumentation inflates these runs, so what the table establishes is the ordering between layouts; the absolute values are not transferable to the uninstrumented campaigns of Chapter 4.	26
5.3	The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is VecScatterBegin and VecScatterEnd; reduction is VecTDot and VecNorm. Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run; the two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of RepeatedSolves KSPSolve time.	31
5.4	Reduction in synchronisation exposure against time per iteration, both relative to flat MPI at the same size–node point. The break-even value is the exposure reduction the fitted line requires to offset threading overhead. a and b are that line’s slope and intercept, so $(1 - b)/a$ reproduces the last column. Eleven points per layout; the fits are descriptive only.	33
A.1	2D strong-scaling campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.	42
A.2	3D strong-scaling campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.	42

List of Figures

2.1	Architecture of an ARCHER2 node, consisting of two 64-core AMD EPYC 7742 processors.	3
4.1	2D time per KSP iteration relative to the 128×1 baseline. Values below 1 mean the hybrid layout is faster.	13
4.2	Median 2D time per KSP iteration along the two unknowns/core chains. Ideal weak scaling is a horizontal line; the rise above each layout's own one-node value is the cost of concurrency, quantified as efficiency in the text and in Table 4.4.	14
4.3	3D time per KSP iteration relative to the 128×1 baseline. Unlike 2D, no series is monotone.	15
4.4	Median 3D time per KSP iteration along the two unknowns/core chains. Ideal weak scaling is again a horizontal line; note that the vertical range is roughly three times the 2D one.	15
4.5	2D time per solve at a fixed 20.25M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1 . The dotted line is that ideal, anchored at the one-node 128×1 point; a curve below it is scaling superlinearly.	18
4.6	2D parallel efficiency at a fixed 20.25M unknowns, each layout against its own one-node time. The dotted line at 100% is ideal strong scaling; above it is superlinear. This ranks scaling, not speed; Figure 4.5 is the like-for-like time comparison.	19
4.7	3D time per solve at a fixed 20.35M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1 ; the dotted line is that ideal, anchored at the one-node 128×1 point. Flat MPI and 64×2 turn and get slower; the most-threaded layouts do not.	20
4.8	3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. The dotted line at 100% is ideal strong scaling. This ranks scaling, not speed; Figure 4.7 is the like-for-like time comparison.	21

5.1	Where the cores are during the solve, from the MAP call trees.	28
5.2	2D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.	30
5.3	3D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Each point is the median of three repetitions. Note the vertical scale: flat MPI reaches 84.6% at the largest point.	30
5.4	3D ExposureRemoved relative to flat MPI against the resulting time per iteration, one point per size–node pair. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. Where a layout's fitted line crosses parity is the break-even column of Table 5.4; the lines are descriptive, not predictive.	33
5.5	2D ExposureRemoved relative to flat MPI against the resulting time per iteration, one point per size–node pair. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. the lines are descriptive, not predictive.	34
A.1	Median 2D repeated-solve throughput along the two unknowns/core chains. The dashed grey line is ideal weak scaling.	41
A.2	Median 3D repeated-solve throughput along the two unknowns/core chains. The dashed grey line is ideal weak scaling.	41

Acknowledgements

Here you can give credit to anyone who has enabled the progress of your MSc project, especially if this was *above and beyond* their role. Below is our acknowledgement of the people who contributed to creation of this template.

This template is a slightly modified version of the one developed by Prof. Charles Duncan for MSc students in the Dept. of Meteorology. His acknowledgement follows:

This template has been produced with help from many former students who have shown different ways of doing things. Please make suggestions for further improvements.

Chapter 1

Introduction

1.1 Motivation

1.2 Aims and Research Questions

This study measures PETSc rank–thread configurations for a Poisson problem on ARCHER2: flat MPI against hybrid MPI+OpenMP, in 2D and 3D, at four problem sizes, from 1 to 64 nodes. Rather than finding whether hybrid parallelism is the best, It asks whether hybrid execution reduces runtime, which configuration gives the lowest runtime, where the time goes, and what architectural factors explain the observed behaviour.

RQ1 Does PETSc’s threaded-kernel backend reduce the time to solution relative to flat MPI on a fully populated node, and by how much?

RQ2 If it does, at which rank–thread balances, and how does the answer change with problem size, node count, dimensionality, and scaling regime?

RQ3 Which parts of the solve account for the difference?

RQ4 How far do the observed trends generalise, and what would be needed to establish a mechanism for them?

1.3 Contributions

1.4 Dissertation Structure

Chapter 2

Background and Related Work

2.1 Parallel Programming Models

A hybrid program assigns one or more MPI ranks to each node and creates an OpenMP team inside every rank.

Table 2.1: The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation; it inherits both cost columns.

Model	Memory	Data exchange	Principal cost
MPI	Private per rank	Explicit messages	Message matching, per-rank solver state, one subdomain boundary per rank
OpenMP	Shared within a team	Shared addresses	Team creation, synchronisation, cache and bandwidth contention
Hybrid	Shared inside a rank, private between ranks	Messages between ranks only	Both, traded against each other by the rank–thread balance

2.2 Limitations of Flat MPI

Flat MPI assigns one rank to each core.

2.3 Hybrid MPI+OpenMP

PETSc –with-openmp-kernels

2.4 ARCHER2 Node Architecture and NUMA

An ARCHER2 compute node has two AMD (Rome) processors, giving 128 physical cores. The service configures four NUMA regions per socket, eight per node, each with 16 cores and local memory channels in each region. A core has private L1 and L2 caches. Four cores forms a Core Complex (CCX) and share a 16MB L3 cache, two CCXs form a Core Complex Die (CCD). Different CCXs do not share L3 cache [1].

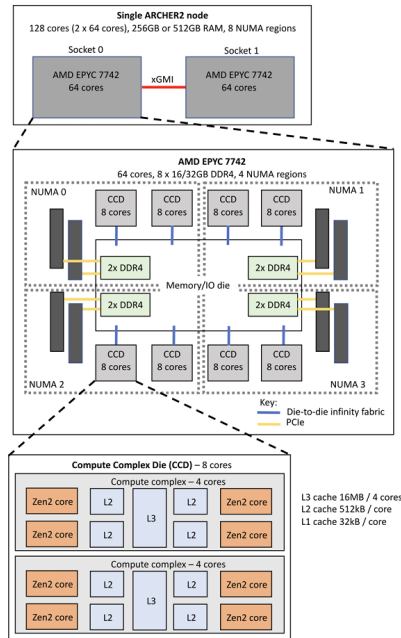


Figure 2.1: Architecture of an ARCHER2 node, consisting of two 64-core AMD EPYC 7742 processors.

NUMA affects each both latency and bandwidth. Linux normally allocates a page in the NUMA region of the thread that first touches it. If a thread creates a large array and a team accesses it later later from several regions, remote traffic can concentrate on the memory controllers of the first region. The ARCHER2 guide also notes that two threads can approach the memort-bandwidth limit of a NUMA region for bandwidth-intensive code [2]. More threads do not guarantee more bandwidth.

Cache topology introduces a finer boundary than NUMA. Four adjacent cores share an L3 cache, whereas a 16-core NUMA region countains four such L3 caches. With

Table 2.2: The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores.

Layout	Cores/rank	Subdomains/node	Rank against the shared-L3 (CCX) boundary
128×1	1	128	four ranks share one CCX
64×2	2	64	two ranks share one CCX
32×4	4	32	exactly one CCX
16×8	8	16	spans two CCXs

close binding, 32×4 can place one four-thread rank within on CCX; 64×2 can use two two-thread rank to share one CCX. 16×8 can make a rank span two CCXs, and 128×1 can make four ranks share one CCX. This mapping is a plausible mechanism for layout differences, The experiment fix thread affinity and compare measuremetns before using topology to interpret the results, The relevant binding options are:

```
--cpu-bind=cores
--distribution=block
--hint=nomultithread
```

2.5 PETSc and Krylov Solvers

PETSc provides distributed vectors and matrices, Krylov solvers (KSP), preconditioners (PC), and a built-in logging system. It talks between processes with MPI, and it lets the user pick solver components at run time.

The production system is the finite-difference Laplacian, whose matrix is a sparse, symmetric, and positive definite. Conjugate gradients (CG) is designed for this type of linear system. At each iteration CG applies the matrix and the preconditioner, updates vectors, and evaluates inner products. Table 2.3 classifies the work of one CG step accordingly. The counts show why one step is dominated by neighbour exchange rather than by the two CG dot products: the multigrid hierarchy performs its own halo exchange on every level, so the 42.9 exchanges per step is more lager than the 3 golbal reductions. Neighbour and global costs scale differently with the rank count, The Section 5.4 separates them and measures them both.

A preconditioner reshapes the spectrum of the system, so CG reaches the requested tolerance in fewer iterations. Algebraic multigrid (AMG) builds a hierarchy of coarser problems directly from the matrix. On each level a smoother quickly removes high-frequency error; the smooth error that a local smoother handles slowly is instead restricted to a coarser level, corrected there, and prolonged back. PETSc's GAMG (geometric-algebraic multigrid, PCGAMG) builds this hierarchy without needing a geometric description of the mesh.

Table 2.3: Operations in one GAMG-preconditioned CG step, with the PETSc event that records each and the number of calls per step measured in a representative run. A step is one preconditioner application.

Operation	PETSc event	Communication	Calls/step
Matrix-vector product	MatMult	neighbour	28.9
Multigrid restriction	MatMultTranspose	neighbour	7.0
Multigrid prolongation	MatMultAdd	neighbour	7.0
Level residual	MatResidual	neighbour	7.0
Halo exchange, all of the above	VecScatterBegin	neighbour	42.9
Inner products	VecTDot	global	2.0
Residual norm	VecNorm	global	1.0
Vector updates	VecAXPY, VecAYPX	none	44.6

2.6 Performance Analysis Tools

`log_view` and Linaro MAP are the two PETSc tools used in this study. `log_view` is a built-in logging system that can report the time spent in each PETSc function, while Linaro MAP is a sampling profiler for MPI, OpenMP, and single-threaded programs that provides detailed information about the performance of PETSc applications, including memory usage, communication patterns, and computational efficiency.

2.7 Summary and Research Gap

2.8 Dissertation Report

Chapter 3

Experimental Platform and Methodology

3.1 Hardware and Software Environment

All experiments ran on ARCHER2, the UK national supercomputing service [3]. Each node has two AMD EPYC 7742 (Rome) processors, 128 physical cores, and eight NUMA regions of 16 cores each [1]. Section 2.4 gives the cache hierarchy below that. Every run disables simultaneous multithreading, so the 128 physical cores are used and their hardware siblings idle. The software stack was PrgEnv-gnu 8.4.0, gcc 11.2.0, and cray-mpich 8.1.27.

PETSc 3.24.4 was built from source. The baseline build, `arch-omp-opt`, was configured as

```
./configure --PETSC_ARCH=arch-omp-opt --with-debugging=0 \  
COPTFLAGS=-O3 CXXOPTFLAGS=-O3 FOPTFLAGS=-O3 \  
--with-cc=cc --with-cxx=CC --with-fc=ftn \  
--with-openmp=1 --with-openmp-kernels=1
```

The Cray compiler wrappers `cc`, `CC` and `ftn` supply MPI, so no MPI path is given explicitly. `-with-openmp-kernels` was added in PETSc 3.22 and enables threading inside PETSc's own kernels [4]; `-with-openmp=1` alone leaves the solve serial within each rank. Both are build-time options, so a flat-MPI run and a hybrid run use the same binary and differ only in the runtime layout. One thread per rank in this build is the MPI-only baseline used throughout.

A second build was added for the following reason. A sparse Krylov solve reaches the dense Basic Linear Algebra Subprograms (BLAS), the standard interfaces for dense vector–vector, matrix–vector and matrix–matrix operations, only incidentally. Any BLAS call it does make would use threads drawn from the same per-rank thread

count as PETSc's own, so an unthreaded BLAS is a possible confound for the layout comparison. The MAP profiles of the baseline build sample no BLAS frame at all (Table 5.2). The production build `arch-omp-libsci-rome-opt` therefore repeats the configure line above with one addition,

```
LIBSCI=/opt/cray/pe/libsci/23.09.1.1/GNU/10.3/x86_64/lib
./configure ... --with-blaslapack-lib=$LIBSCI/libsci_gnu_mp.so.5.0
```

which links PETSc against the threaded Cray LibSci library. LibSci is HPE Cray's tuned implementation of BLAS and LAPACK; the `_mp` suffix marks the variant whose kernels are threaded with OpenMP. All repeated-solve campaigns of Chapter 4 and Chapter 5 use this build. Section 5.3 measures how much of the solve the threaded library accounts for.

3.2 Benchmark Problems and Drivers

The 2D benchmark is based on PETSc's KSP tutorial `ex2` (`src/ksp/ksp/tutorials/ex2.c` in the PETSc source tree [5]). This implements a five-point stencil discretisation of the Laplace operator on an $m \times n$ grid, giving an $mn \times mn$ sparse matrix with five nonzeros per row. The 3D benchmark, `3d_repeat`, is a driver written for this project. It assembles the seven-point stencil on an $m \times n \times p$ grid, with seven nonzeros per row and otherwise the same structure. It was added because the surface-to-volume ratio of a subdomain, and hence the fraction of its rows that couple off-rank, differs between the two dimensionalities.

Both benchmarks are run through a repeated-solve driver rather than solved once. The 2D driver `ex2_repeat` separates one warm-up solve from 19 timed solves inside a dedicated PETSc logging stage, with the preconditioner reused across the timed solves; `3d_repeat` applies the same design in 3D. Section 5.1 gives the reason and the measured cost split that motivates it. Everything timed in Chapter 4 is a steady-state solve in that sense.

3.3 Solver Configuration

The default `ex2` solver is GMRES with block Jacobi. Block Jacobi builds one block per MPI rank, so the preconditioner is a function of the rank decomposition: changing the rank-thread layout changes the operator that is applied. Table 3.1 shows this directly. Reading across any row, the iteration count does not change at all when threads are added; reading down any column, it changes by a factor of more than three, from 2825 at four ranks to 10000 at 128.

That column is also not monotonic in the rank count: it falls from 6700 at one rank to 2825 at four, then rises again to 8179 at 16 before falling to 6460 at 32. More ranks

therefore neither strengthen nor weaken the preconditioner in any consistent direction, so the dependence cannot be read as a trade-off between preconditioner strength and parallelism. Two layouts with different rank counts apply different preconditioners to the same linear system, and a wall-clock difference between them reflects that difference as much as any property of the layout. Cross-layout runtime under this solver is therefore indicative only, and the question “which rank–thread layout is fastest?” has no well-posed answer under it.

Table 3.1: Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.

MPI ranks	OpenMP threads per rank							
	1	2	4	8	16	32	64	128
1	6700	6700	6700	6700	6700	6700	6700	6700
2	3715	3715	3715	3715	3715	3715	3715	
4	2825	2825	2825	2825	2825	2825		
8	4085	4085	4085	4085	4085			
16	8179	8179	8179	8179				
32	6460	6460	6460					
64	7181	7181						
128	10000							

The production campaigns therefore replace that solver. Every formal run uses

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

The finite-difference Laplacian is symmetric and positive definite, which is the class of system conjugate gradients was designed for [6], and CG is the standard Krylov method for that class in PETSc [7, 8]. PETSc’s GAMG is an algebraic multigrid preconditioner. It builds a hierarchy of coarse operators from the matrix itself and uses them to remove the low-frequency error that a one-level method removes slowly [9, 7]. Its convergence is governed mainly by that hierarchy rather than by how the matrix rows are distributed, so its iteration count depends more weakly on the rank decomposition than block Jacobi’s does. It is not independent of it: coarsening is built from the local matrix, so the hierarchy is not invariant under repartitioning.

The dependence is therefore reported as a measurement. Across the 11 size–node points of each dimension in Chapter 4, the four layouts differ by at most one iteration at every point except 2D 5m on one node, where they span 9 to 11. Block Jacobi spans a factor of 3.5 in Table 3.1 on a single problem. Every figure in Chapter 4

is reported per iteration, so the residual one-iteration differences cannot be read as layout effects.

A GAMG-preconditioned solve pays its cost in two places. PCSetUp builds the hierarchy once; PCApply applies it inside every CG iteration, and PETSc reports both under the enclosing KSPSolve. Without preconditioner reuse each call to KSPSolve rebuilds the hierarchy first, so the call structure is

```
KSPSolve
  PCSetUp                (hierarchy construction)
  CG iteration loop
    PCApply              (hierarchy application)
```

With `-ksp_reuse_preconditioner` the PCSetUp branch is taken on the first solve only, and later solves enter the loop directly. The two phases carry different costs, and a single-solve measurement reports mostly the setup one. The production campaign therefore times repeated solves with the hierarchy reused. Section 5.1 gives the measured split, the driver that produces it, and the `-log_view` evidence for this call structure.

3.4 Configuration Space and Selection Rules

A full sweep of every rank–thread combination at every problem size and node count is larger than the available allocation. Three rules reduce it to the configurations the formal campaigns run. The first is measured; the second and third are chosen.

Rule 1: threads per rank are restricted to $\{1, 2, 4, 8\}$. This rule comes from the single-node exploratory sweep of the project preparation phase, which ran every ranks $\times$ threads = 128 combination on a 1000^2 grid. Cost per KSP iteration falls monotonically as cores are spent on ranks rather than threads: 0.38 ms at 128×1 , 0.61 at 64×2 , 0.99 at 32×4 , 1.89 at 16×8 , 3.67 at 8×16 , and 31.10 at 1×128 (Table 3.1 gives the iteration counts of the same runs). The step from eight threads per rank to sixteen roughly doubles the per-iteration cost again, and no configuration with 16 or more threads per rank comes within a factor of two of the best. Sixteen threads is also where a rank first fills a whole 16-core NUMA region. The restriction is a search bound read off measurement, not a claim about a NUMA or cache mechanism.

Rule 2: nodes are fully populated. $\text{ppn} \times \text{threads} = 128$, giving the four layouts 128×1 , 64×2 , 32×4 , and 16×8 . Same node count then means same core count, so a comparison between layouts isolates rank–thread balance from core count.

Rule 3: work per core is held inside a fixed window. A configuration at size N on C physical cores is included when $25,000 \leq N/C \leq 100,000$. The lower bound keeps each rank's subdomain large enough that the solve is not dominated by fixed per-call overheads; the upper bound keeps the largest cases inside the memory available per node. The completed 2D campaign uses six grids: 2240^2 , 3160^2 , 4500^2 , 6400^2 , 9073^2 , and 12828^2 , or 5.02M to 164.56M unknowns, and the 3D campaign uses the matching cubes of Table 4.1. The filter admits 11 size–node pairs over 1–32 nodes in each dimension.

Rule 3 has one deliberate exception. The strong-scaling campaign of Section 4.1.2 drives the 20M problem out to 32 nodes, which leaves about 4.9 thousand unknowns per core, a factor of five below the window. That campaign looks for the node count at which each layout stops scaling, which lies outside the window by construction. The 32-node point is a saturation tail, not a recommended operating point, and no weak-scaling claim is made from it.

3.5 Job Configuration and Thread Placement

Each rank and its threads are pinned to a contiguous set of cores with:

```
OMP_PLACES=cores    OMP_PROC_BIND=close
srun --hint=nomultithread --distribution=block:block --exact
```

The primary timing is the maximum-rank KSPSolve time in the RepeatedSolves stage. If this event has count $c = 19$, time t , and the grid has N unknowns, steady-state throughput is

$$Q = \frac{Nc}{t}.$$

The analysis also reports $t/(cI)$, where I is the final KSP iteration count. Every configuration is run three times. The three repeats are aggregated by their median. The error bars in every figure span the minimum and maximum of those three runs, so they are the full observed range and not a standard error; three samples do not support a distributional estimate. Scaling is computed per layout against that layout's smallest admitted node count. For a layout ℓ at n nodes,

$$S_\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E_\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

where n_0 is the smallest node count run for that problem size.

Chapter 4

2D and 3D Problems

4.1 Experimental Setup

Two campaigns cover each dimension. Both use the platform, build, solver and thread placement of Chapter 3, and both fill every node with all 128 cores. They differ only in how the problem size responds to the node count. The weak-scaling campaign grows the problem in proportion to the nodes, holding the work per core fixed. The strong-scaling campaign holds one problem size fixed and spreads it over more nodes. Table 4.1 lists both.

Table 4.1: Grids and node counts of the six scales. The strong-scaling column is not limited by the unknowns-per-core filter of Section 3.4; its 32-node point sits deliberately below the window.

Scale	2D grid	3D grid	Weak-scaling nodes	Strong-scaling nodes
5m	2240^2	171^3	1	—
10m	3160^2	215^3	1, 2	—
20m	4500^2	273^3	2, 4	1, 2, 4, 8, 16, 32
41m	6400^2	345^3	4, 8	—
82m	9073^2	435^3	8, 16	—
165m	12828^2	548^3	16, 32	—

The scale labels are nominal. 2240^2 is 5,017,600 unknowns against $171^3 = 5,000,211$ in 3D, and the two dimensions match to within 0.5% at every scale, so a 2D and a 3D row of the same label solve systems of the same size.

That gives 11 size-node pairs in the weak-scaling campaign and six node counts in the strong-scaling one. Every pair runs all four layouts three times. All 280 weak-scaling logs and all 144 strong-scaling logs passed the content checks. None carries a

divergence, a PETSc or MPI error, or a licence wait.

Run-to-run spread sets the resolution of every margin quoted below. Define the spread of a configuration as $(\max - \min)/\text{median}$ over its three repeats of the solve time. The largest spread anywhere in the weak-scaling matrix is 4.71% in 2D and 3.83% in 3D. For the strong-scaling curves it is 10.77% and 11.09%, because a solve at the largest node counts takes only tens of milliseconds and a fixed per-call overhead is then a visible fraction of it. A difference between two layouts smaller than these figures is reported as a tie: the measurement cannot separate them, which is not the same as their being equally fast.

4.1.1 Weak-Scaling Chains

The filter $25,000 \leq N/C \leq 100,000$ admits each scale at only one or two node counts, so a panel drawn per problem size holds two points. The same 11 pairs split into two chains of near-constant unknowns per core (Table 4.2). Ideal weak scaling would hold the time per iteration flat along a chain. The measured rise is the cost of concurrency.

Table 4.2: The 11 admitted size–node pairs, ordered into two weak-scaling chains.

Band	Size at node count					
	1	2	4	8	16	32
~40k unknowns/core	5m	10m	20m	41m	82m	165m
~80k unknowns/core	10m	20m	41m	82m	165m	—

4.1.2 Strong-Scaling Design

The chains span one node doubling per problem size. That cannot show where a layout stops scaling. A second campaign therefore fixes the 20m problem and runs the full sequence $\{1, 2, 4, 8, 16, 32\}$ nodes, with nothing else changed. At 32 nodes this leaves about 4.9 thousand unknowns per core, below the admission window. That point is the saturation tail and not a sensible operating point.

4.2 Weak Scaling at Constant Work per Core

Both dimensions run the same 11 pairs, the same four layouts and the same three repeats.

4.2.1 2D

Throughput rises almost linearly with the node count (Figure A.1). All four layouts sit within a few per cent of each other on that axis, so it cannot separate them. Figure 4.1 divides each layout's time per iteration by that of flat MPI at the same size–node pair. This removes the scale and the GAMG iteration count and leaves the layout effect.

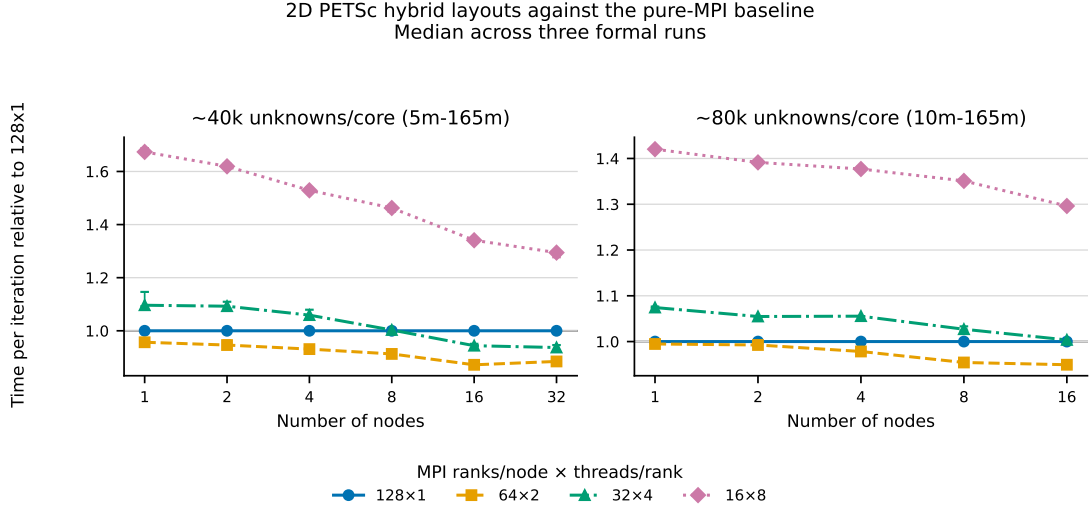


Figure 4.1: 2D time per KSP iteration relative to the 128×1 baseline. Values below 1 mean the hybrid layout is faster.

The curves fall. Along the $\sim 40k$ chain 64×2 goes from 0.957 at one node to 0.872 at 16, and 32×4 from 1.096 to 0.937. Along the $\sim 80k$ chain all three hybrid ratios drop at every step. Four of the six series are strictly monotone.

Two things follow. 64×2 beats flat MPI per iteration at all 11 pairs, and its lead grows with the node count. 32×4 starts 9.6% behind flat MPI, reaches parity at eight nodes, and is 6.3% ahead at 32 nodes. A ranking taken at one node does not hold at 32.

16×8 is the slowest layout at every pair. Its deficit does narrow, from 46.2% at 5m on one node to 22.7% at 165m on 32 nodes.

Figure 4.2 gives the absolute time per iteration. A flat line would be ideal weak scaling.

No line is flat, so no layout weak-scales perfectly. Stated as weak-scaling efficiency $E_\ell(n) = t_\ell(n_0)/t_\ell(n)$ along a chain, the $\sim 40k$ chain runs from 100% at one node to 59.6%, 64.5%, 69.8% and 77.1% at 32 nodes for 128×1 , 64×2 , 32×4 and 16×8 . The $\sim 80k$ chain spans one doubling fewer and gives each core twice the work; it holds 81.9%, 85.8%, 87.7% and 89.8% at 16 nodes. Two things follow. Efficiency falls at every doubling on every chain and every layout, so the cost of concurrency is

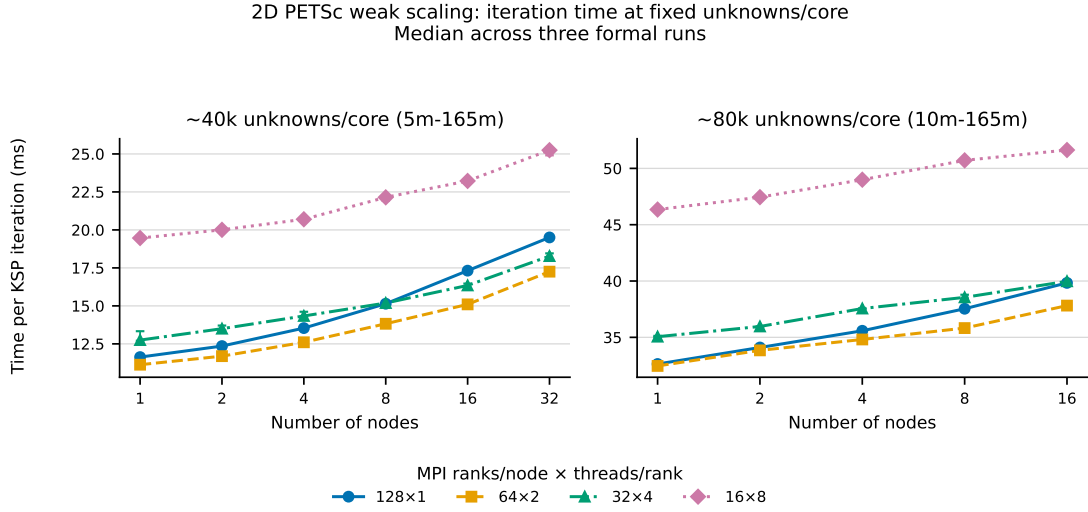


Figure 4.2: Median 2D time per KSP iteration along the two unknowns/core chains. Ideal weak scaling is a horizontal line; the rise above each layout’s own one-node value is the cost of concurrency, quantified as efficiency in the text and in Table 4.4.

paid gradually and not at one threshold. It also falls more slowly the more threads a rank has, in strict order, at both ends of both chains. Table 4.4 gives the endpoints; Section 4.2.3 explains why that ordering is not a recommendation.

4.2.2 3D

The 3D campaign uses the same points, layouts and repeats. Its throughput has the same near-linear shape (Figure A.2), so the comparison again uses the ratio (Figure 4.3).

3D differs from 2D in size and in shape. The best point is 32×4 at 41m on four nodes, at a ratio of 0.751. That is 24.9% less time per iteration than flat MPI, against a 2D best of 12.8%. Each dimension has 33 hybrid measurements, the 11 size–node pairs times the three hybrid layouts. In 3D, 22 of the 33 fall below 1; in 2D, 13 of 33 do.

The shape differs too. None of the six series is monotone. On the $\sim 80k$ chain the hybrid layouts lead at two and four nodes, lose the lead by eight nodes, and are at or above parity at 16. On the $\sim 40k$ chain 64×2 rises to 1.022 at 41m on eight nodes and then falls back to 0.768 at 165m on 32.

16×8 wins no point. Unlike the 2D case it is not always last, and it beats flat MPI at five of the 11 pairs.

Figure 4.4 gives the absolute time per iteration, on the same convention as the 2D case.

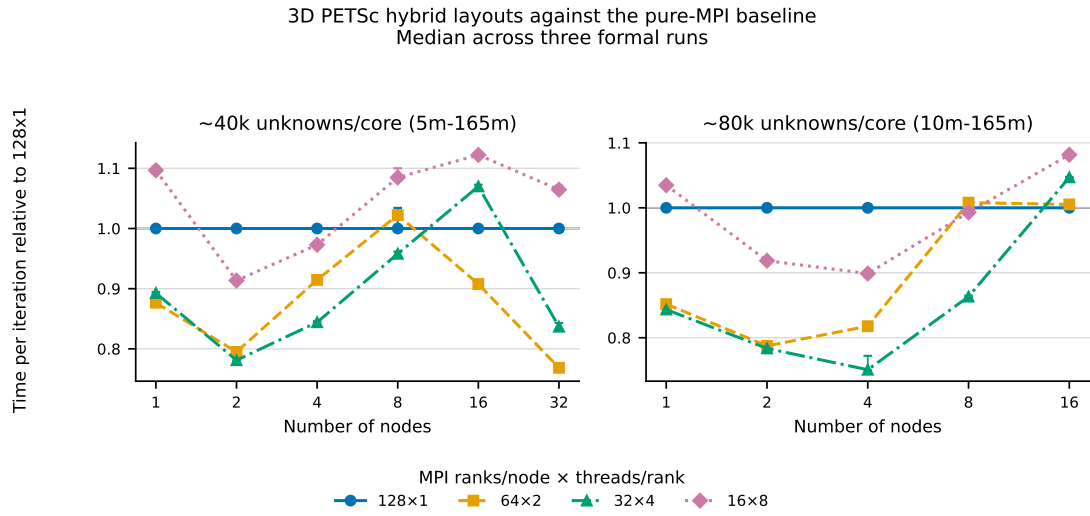


Figure 4.3: 3D time per KSP iteration relative to the 128×1 baseline. Unlike 2D, no series is monotone.

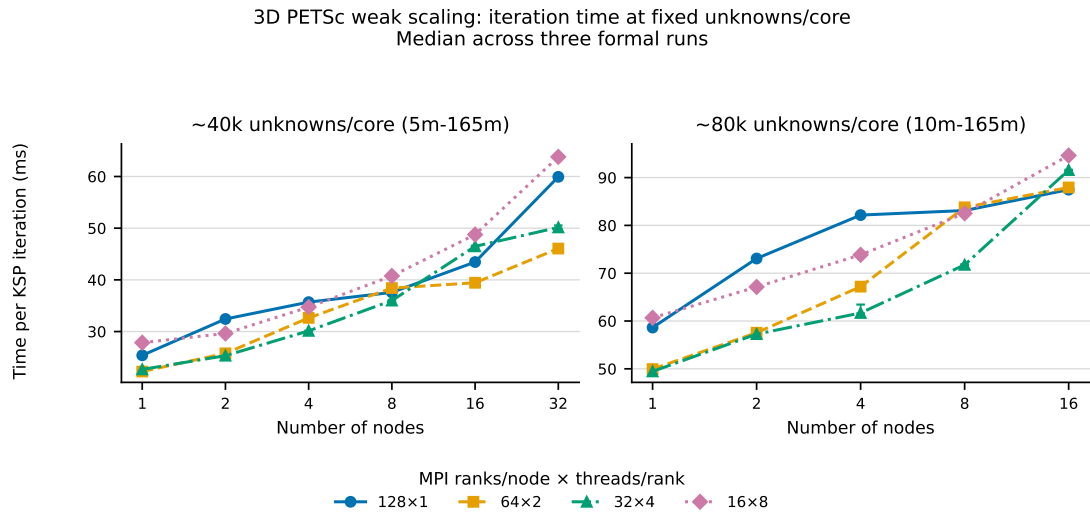


Figure 4.4: Median 3D time per KSP iteration along the two unknowns/core chains. Ideal weak scaling is again a horizontal line; note that the vertical range is roughly three times the 2D one.

The departure from flat is larger than in 2D. On the $\sim 40k$ chain the four layouts end at 42.4%, 48.3%, 45.2% and 43.7% weak-scaling efficiency at 32 nodes, against 59.6–77.1% in 2D. On the $\sim 80k$ chain they end at 67.0%, 56.8%, 54.0% and 64.1% at 16 nodes, against 81.9–89.8%. The 3D loss over the same node range is therefore roughly a third larger. It also carries no ordering: the best-scaling layout is 64×2 on one chain and 128×1 on the other, where in 2D 16×8 is best on both. The term that grows with node count in 3D is not a simple function of the thread count. Chapter 5 measures it.

4.2.3 Layout Ranking and Efficiency

Table 4.3 gives the fastest layout at every pair and its margin over flat MPI.

Table 4.3: Fastest full-node layout at every admitted size–node pair, by median throughput. A margin of 0.0% means flat MPI was fastest.

Scale	Nodes	Cores	Best layout	M eq/s	vs. MPI	ms/iteration
<i>2D, five-point stencil</i>						
5m	1	128	128×1	47.9	0.0%	11.63
10m	1	128	128×1	34.0	0.0%	32.63
10m	2	256	64×2	94.9	5.7%	11.69
20m	2	256	64×2	59.8	0.8%	33.84
20m	4	512	64×2	160.7	7.4%	12.60
41m	4	512	64×2	107.0	2.2%	34.81
41m	8	1024	128×1	270.5	0.0%	15.14
82m	8	1024	64×2	208.9	4.8%	35.82
82m	16	2048	64×2	495.6	14.7%	15.10
165m	16	2048	64×2	362.6	5.3%	37.82
165m	32	4096	64×2	794.8	13.1%	17.25
<i>3D, seven-point stencil</i>						
5m	1	128	64×2	37.4	14.1%	22.26
10m	1	128	32×4	28.7	1.6%	49.45
10m	2	256	64×2	64.3	25.8%	25.78
20m	2	256	32×4	50.8	27.6%	57.26
20m	4	512	32×4	96.5	1.5%	30.13
41m	4	512	32×4	95.1	33.2%	61.68
41m	8	1024	32×4	163.0	4.4%	35.99
82m	8	1024	32×4	163.9	15.8%	71.75
82m	16	2048	64×2	298.2	10.2%	39.44
165m	16	2048	128×1	268.8	0.0%	87.47
165m	32	4096	64×2	510.4	30.1%	46.06

64×2 wins 8 of the 11 2D pairs. In 3D 32×4 wins 6, 64×2 wins 4 and flat MPI wins 1. The 2D winner changes twice down the column and the 3D winner changes five times. 2D has one layout ahead throughout. 3D does not. One of the three 2D flat-MPI wins, 41m on eight nodes, is a tie: the medians differ by 0.4% and the ranges overlap.

Table 4.4 measures each layout against its own one-node time. This is the standard weak-scaling efficiency, but it ranks scaling and not speed, and the two can disagree. A layout that starts slow has an easy baseline to hold, so it can post the best efficiency while running the worst time at every point.

Table 4.4: Weak-scaling efficiency (%) at the largest node count of each chain, against each layout’s own one-node time.

Case	Chain and node count	Layout			
		128×1	64×2	32×4	16×8
2D	$\sim 40k$, 32 nodes	59.6	64.5	69.8	77.1
2D	$\sim 80k$, 16 nodes	81.9	85.8	87.7	89.8
3D	$\sim 40k$, 32 nodes	42.4	48.3	45.2	43.7
3D	$\sim 80k$, 16 nodes	67.0	56.8	54.0	64.1

In 2D the ordering is strict in both chains. Efficiency rises with threads per rank, and 16×8 degrades least of the four. In 3D there is no such ordering. On the $\sim 40k$ chain 64×2 degrades least, and on the $\sim 80k$ chain flat MPI does.

The 2D ordering is that case. 16×8 has the best 2D efficiency and the worst 2D time at every one of the 11 pairs: 19.47 ms per iteration at 5m on one node against 11.63 ms for flat MPI, and 25.25 against 19.50 at 165m on 32 nodes. It degrades least because its cost is already dominated by a term that does not grow with the node count, so the growing term is a smaller fraction of a larger number. Efficiency measured against a layout’s own baseline rewards a bad baseline. Every layout comparison in this chapter is therefore made on time per iteration, and efficiency is used only to describe how a layout scales against itself.

4.3 Strong Scaling at Fixed Problem Size

4.3.1 2D

The 2D curves scale well and stay together (Figure 4.5). Every layout is still improving at 32 nodes. The spread between fastest and slowest goes from $1.47\times$ at one node to $1.81\times$ at 16 and back to $1.48\times$ at 32. 64×2 and flat MPI finish within 2% of each other. 16×8 starts 47% behind and ends 45% behind.

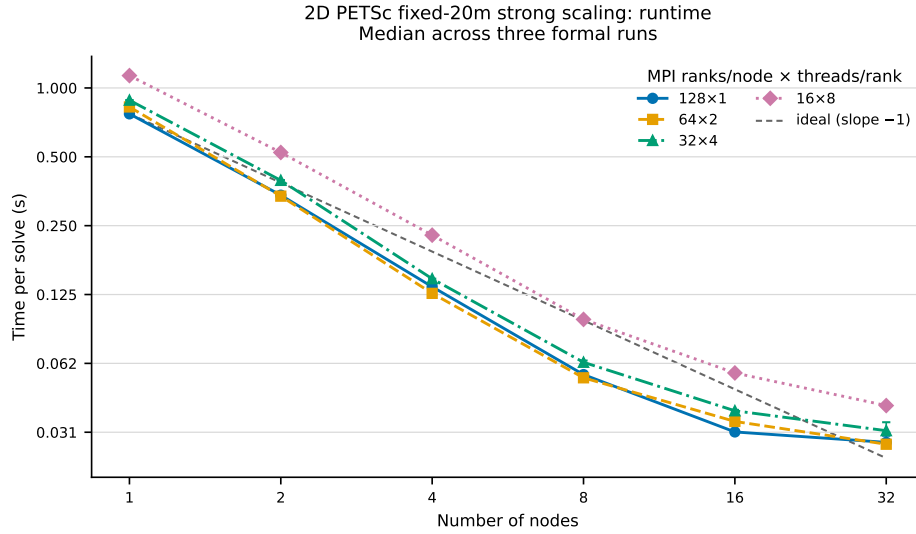


Figure 4.5: 2D time per solve at a fixed 20.25M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1 . The dotted line is that ideal, anchored at the one-node 128×1 point; a curve below it is scaling superlinearly.

Cumulative efficiency passes 100% between two and 16 nodes and peaks at 191% for 64×2 at eight nodes. These superlinear values repeat across all three runs, so they are not a single anomalous measurement.

The usual explanation for superlinear strong scaling is that the working set shrinks into a faster level of the memory hierarchy as the fixed problem is divided over more caches. The arithmetic here is consistent with that. A five-point row costs about 60 bytes of matrix storage, so a flat-MPI rank holding 158 thousand unknowns at one node carries about 9.5 MB, and the four ranks sharing one 16 MB CCX ask for about 38 MB of it. By 32 nodes the same rank holds about 4.9 thousand unknowns, or 0.3 MB, and the four together ask for 1.2 MB. Somewhere in that range the matrix stops being streamed from memory on every application. The vectors and the GAMG hierarchy shift the same way.

This experiment cannot test that explanation. It reports wall-clock time and PETSc event times; neither `-log_view` nor a MAP call tree measures cache occupancy, hit rate, or memory traffic, so nothing in the data separates a cache effect from any other cause of superlinearity. The data do show that the effect is not uniform across layouts. Every layout peaks at eight nodes, at 191% for 64×2 , 175% for 32×4 , 173% for flat MPI, and 145% for 16×8 . 16×8 gives its ranks the largest subdomains and so the largest working set per L3, and it is the least superlinear of the four, which is the direction a cache account predicts. Confirming it needs hardware counters for L3 misses and memory bandwidth. Chapter 7 sets out that measurement.

Figure 4.6 shows the same runs as parallel efficiency, each layout against its own one-node time, so it ranks scaling and not speed. Every layout rises above 100%, peaks at eight nodes, and falls back through 100% between 16 and 32 nodes, ending at 85–93%. Together with Figure 4.5 this says that 2D is still scaling at 32 nodes but no longer scaling superlinearly, and that the four layouts do so together.

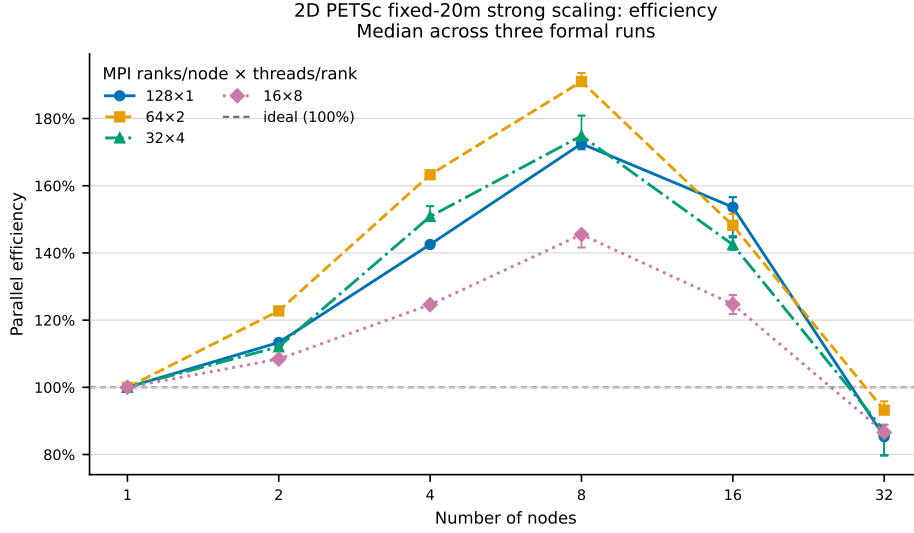


Figure 4.6: 2D parallel efficiency at a fixed 20.25M unknowns, each layout against its own one-node time. The dotted line at 100% is ideal strong scaling; above it is super-linear. This ranks scaling, not speed; Figure 4.5 is the like-for-like time comparison.

4.3.2 3D

The 3D result is different in kind, and it is the largest layout effect in this study (Figure 4.7). Every 3D layout eventually turns and gets slower. They turn at different node counts, and the order is the thread count.

Flat MPI turns first. It bottoms at eight nodes with 0.0997 s per solve, then rises to 0.1776 s at 16 nodes and 0.2599 s at 32. 64×2 and 32×4 bottom at 16 nodes. 16×8 has not turned by 32 nodes. There it is the fastest of the four at 0.0681 s per solve, $3.81\times$ faster than flat MPI on the same 4096 cores. The three repeats of 16×8 at that point span 0.0672–0.0724 s and the three repeats of flat MPI span 0.2594–0.2624 s, so the ordering is not a sampling artefact.

Call the node count at which a layout's time stops falling its saturation point: the point at which the communication and synchronisation cost of the extra ranks has caught up with the compute time they remove. That point rises with threads per rank: 8 nodes for 128×1 , 16 for 64×2 , 16 for 32×4 , and beyond 32 for 16×8 .

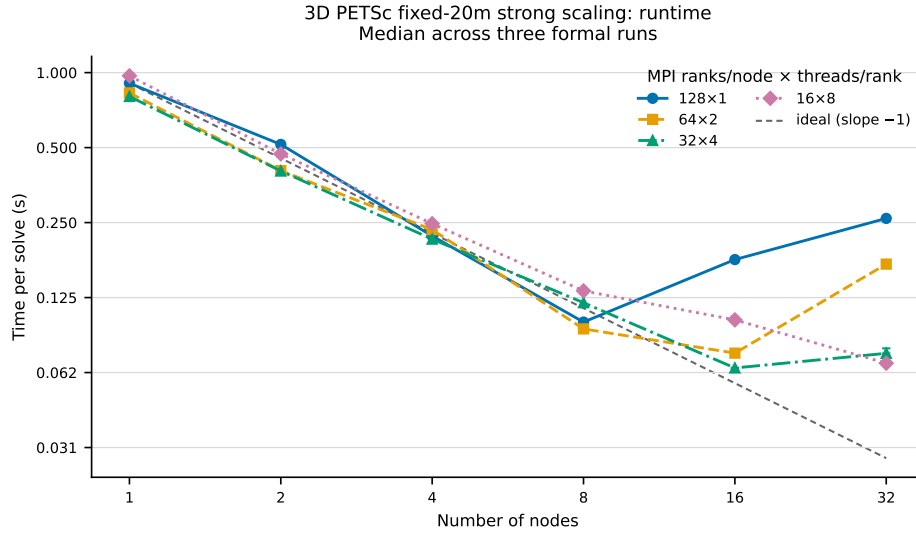


Figure 4.7: 3D time per solve at a fixed 20.35M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1 ; the dotted line is that ideal, anchored at the one-node 128×1 point. Flat MPI and 64×2 turn and get slower; the most-threaded layouts do not.

Replacing ranks with threads does not make the solve faster at the node counts where flat MPI is still scaling. It moves the node count at which scaling stops. This is why the 3D advantage in Table 4.5 is small up to eight nodes and large beyond 16.

Figure 4.8 is the counterpart of Figure 4.6. Each layout is measured against its own one-node time, so it ranks scaling and not speed. In 2D all four layouts rise above 100% and land within eight points of each other at 32 nodes. In 3D they separate by a factor of four, from 10.9% for flat MPI to 44.5% for 16×8 , and they fall from their peak earlier. That fall is the saturation of the runtime plot expressed as a fraction of ideal.

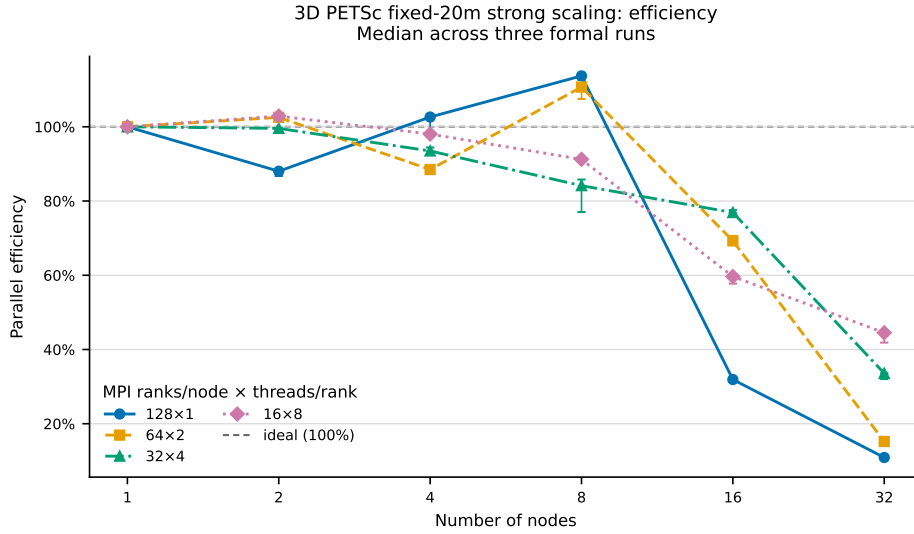


Figure 4.8: 3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. The dotted line at 100% is ideal strong scaling. This ranks scaling, not speed; Figure 4.7 is the like-for-like time comparison.

4.3.3 The 32-Node Endpoint

Table 4.5 gives the fastest layout at each node count. In 2D the margin over flat MPI never exceeds 6.9%. In 3D it reaches 172.4% at 16 nodes and 281.3% at 32.

Table 4.5: Fastest full-node layout at each node count for the fixed 20M problems, with its advantage over flat MPI.

Nodes	Cores	2D, 4500 ²			3D, 273 ³		
		Best	s/solve	vs. MPI	Best	s/solve	vs. MPI
1	128	128 × 1	0.7704	0.0%	32 × 4	0.8022	13.0%
2	256	64 × 2	0.3365	1.0%	32 × 4	0.4028	27.8%
4	512	64 × 2	0.1265	6.9%	32 × 4	0.2145	2.9%
8	1024	64 × 2	0.0541	3.3%	64 × 2	0.0936	6.5%
16	2048	128 × 1	0.0313	0.0%	32 × 4	0.0652	172.4%
32	4096	64 × 2	0.0277	1.9%	16 × 8	0.0681	281.3%

Table 4.6 gives all four layouts at 32 nodes on two measures. Time per solve compares layouts directly. Speedup compares each layout only with itself at one node.

In 2D the two disagree. The four speedups are almost equal, 27.3 to 29.8, but the runtimes behind them differ by 45%, from 0.0277 s for 64 × 2 to 0.0409 s for 16 × 8.

16×8 divides by the worst one-node time in the table, 1.1325 s against 0.7704 s for flat MPI, so it is credited for recovering ground the other layouts never lost. In 3D the two agree. 16×8 has the largest speedup, 14.24 against 3.49 for flat MPI, and the fastest time, 0.0681 s against 0.2599 s. They agree because the 3D layouts diverge as nodes are added rather than converging, so a layout's rank no longer depends on which baseline it is divided by. Only the time column ranks layouts; the speedup column describes how each one scales.

Table 4.6: The 32-node endpoint of the strong-scaling curves. Only time per solve compares layouts directly.

Case	Layout	s/solve	Speedup	Efficiency	16→32 step
2D	128×1	0.0282	27.30	85.3%	55.5%
2D	64×2	0.0277	29.83	93.2%	62.9%
2D	32×4	0.0317	27.85	87.0%	61.1%
2D	16×8	0.0409	27.71	86.6%	69.4%
3D	128×1	0.2599	3.49	10.9%	34.2%
3D	64×2	0.1703	4.87	15.2%	22.0%
3D	32×4	0.0747	10.75	33.6%	43.7%
3D	16×8	0.0681	14.24	44.5%	74.6%

4.4 Interpretation

Table 4.7 collects the comparison. It gives both a throughput gain and a time saving, because these are different conventions and the chapter quotes both. The two halves do not carry equal weight, and the reason is measurement resolution rather than a preference for one dimension. The 3D advantage over flat MPI at 32 nodes is $3.81\times$, against a worst-case run-to-run spread of 11.09% in that campaign, so it is a result. The margins the 2D layouts hold on the same problem are 1–7%, inside or barely outside the 10.77% 2D spread, so they are not. The 2D ordering at any single node count is indicative only; what carries weight in 2D is the trend across node counts, where every point moves the same way. The 3D solve is sensitive to the rank–thread balance and the 2D solve is not, at the resolution available here.

2D is ordered and 3D is not. Every 2D hybrid layout improves against flat MPI as concurrency grows, and weak efficiency rises with threads per rank. In 3D the margins are about twice as large, but no series is monotone and the best layout changes with scale and node count.

The regime matters as much as the dimension. At constant work per core the layout effect is a few per cent. At constant problem size it reaches $3.81\times$ by 32 nodes

Table 4.7: The two dimensions on the quantities this chapter compares them by.

	2D	3D
<i>Constant work per core, 11 size–node pairs</i>		
Layout winning most points	64×2 (8 of 11)	32×4 (6 of 11)
Hybrid measurements faster per iteration	13 of 33	22 of 33
Largest throughput gain	14.7%	33.2%
Largest per-iteration saving	12.8%	24.9%
Ratio series monotone in nodes	4 of 6	0 of 6
Weak efficiency rises with threads	yes	no
<i>Fixed 20M problem, one to 32 nodes</i>		
Layouts whose time turns rise	none of 4	3 of 4
First to rise	—	128×1 , at 8 nodes
Fastest at 32 nodes	64×2	16×8
Its lead over flat MPI	$1.02\times$	$3.81\times$
Fastest-to-slowest spread at 32 nodes	$1.48\times$	$3.81\times$

and is still growing, because the layouts differ in where they stop scaling rather than in speed.

The two stencils do different work per equation, so their absolute rates are not a comparison of algorithms. What compares is the layout ordering and the margin over flat MPI.

Two results here are not explained here. The first is the 2D ordering. Adding threads helps more as concurrency grows, which is what a communication-share account would predict. This chapter measures runtime and not communication, so it cannot test that account. The second is 16×8 . It removes the most MPI endpoints of any layout and is still the slowest 2D layout at all 11 points. Whatever hybridisation saves, it also costs something that grows with the thread count. Chapter 5 measures both directly.

What this chapter does settle is that a layout recommendation needs the regime it was measured in. 16×8 is last at constant work per core in both dimensions and first at 4096 cores on the fixed 3D problem, by $3.81\times$. Both readings are right. They answer different questions.

Chapter 5

Profiling and Bottleneck Analysis

5.1 Why a Repeated-Solve Driver

The quantity of interest is the cost of a steady-state solve as a function of the rank–thread layout: repeated application of a fixed operator and a fixed preconditioner, which is what a production code does inside a time step or an outer nonlinear iteration. A benchmark that solves once measures something else. In a single solve at 20.25M with 16×8 , `-log_view` gave `PCSetUp` 45% of the run and `KSPSolve` 10%. Nearly half the measured time built the multigrid hierarchy, which is done once, and a tenth of it went to the iterations whose scaling Chapter 4 studies. Timing such a run would rank the layouts mainly on preconditioner setup. The setup and the iterations also need not respond to the layout in the same direction, so the two costs cannot be separated afterwards from a single number.

`ex2_repeat` runs the solves inside a `RepeatedSolves` PETSc logging stage, zeroes the initial guess before each one so that every solve does the same work, and reuses the preconditioner across them (`-ksp_reuse_preconditioner` and `-pc_gamg_reuse_interpolation true`). That is the reuse branch of the call structure in Section 3.3. One warm-up solve absorbs the hierarchy construction and the first-touch page faults; the 19 solves that follow are timed. Within the stage, `KSPSolve` accounts for almost all of the time, and each of the 19 repeats took the same 11 iterations. Chapter 4 therefore compares steady-state solves, with hierarchy-construction and first-touch costs excluded by construction rather than subtracted.

5.2 Analysis Methodology

The method combines two instruments.

`-log_view` is PETSc’s built-in profiler. Passing the option to a PETSc program makes it print, at exit, a table of every instrumented event (`MatMult`, `VecScatterBegin`,

PCApply and the rest) with the time and floating-point operations each accumulated, broken down by logging stage [7]. It costs almost nothing and needs no special build, so it is available on every formal run. Two of its columns are used here: %T, the share of the stage's time an event took, and %F, the share of the stage's flops it performed. An event that does a tenth of the arithmetic in a tenth of the time is keeping up. An event whose %T exceeds its %F takes longer than its arithmetic accounts for, and the difference goes to communication, synchronisation, or memory traffic.

Linaro MAP 25.0.4 (Section 2.6) is the second instrument. It samples the call stacks of every process over time [10], which shows where in the software stack the cores are. It requires an instrumented run, so it covers only a few configurations.

Table 5.1 lists single-point `-log_view` data for the RepeatedSolves stage at 20.25M with 16×8 .

Table 5.1: Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8 .

Event	%T	%F	GFlop/s
MatMult	39	55	30.5
MatMultTranspose	18	6	6.7
VecTDot+VecNorm	8	6	–
PCApply (agg.)	82	82	21.6

MatMultTranspose (GAMG restriction) has a %T three times its %F and a floating-point rate $4.6\times$ below MatMult. It takes 18% of the stage's time to do 6% of its arithmetic. VecTDot and VecNorm are the CG global reductions. Their row carries no flop rate because it aggregates two events that PETSc logs separately, and a rate formed from summed flops over summed times would not correspond to either kernel. For a global reduction the quantity of interest is not the rate but the time every rank spends waiting for the slowest one, which is the %T of 8% against a %F of 6%.

5.3 MAP Profiling Results

Two MAP campaigns profiled the same 20.25M 2D problem, one on each of the two PETSc builds of Section 3.1:

1. a baseline set on `arch-omp-opt`, the build with no explicitly linked threaded BLAS, at 2, 8 and 16 nodes;
2. a matching set on the production `arch-omp-libsci-rome-opt`, identical except that it links the threaded Cray LibSci, at 2 and 8 nodes.

The pairs at 2 and 8 nodes differ in one build option, so a difference between them is attributable to that option. Of the twelve profiles, eleven are used. The 16-node 16×8 baseline profile is rejected on its content: MAP resolved no library symbols in it, so every sample is attributed to the sampler and the file carries no cost information.

Neither campaign contains a 128×1 cell. The constraint is the Linaro Forge licence on ARCHER2, which caps the total number of processes that may be profiled at one time and shares that cap across all users of the service. It is not an inability of MAP to attach to a particular layout. A flat-MPI run puts 128 ranks on every node, so it exhausts the available process budget at a node count below the ones needed here. Flat MPI therefore has `-log_view` evidence only, and every MAP-based statement in this section is anchored at its hybrid end. The missing point is fixed at zero OpenMP-runtime time by construction, because a one-thread rank has no OpenMP team to wait for. Repeating the matrix with a larger share of the licence is listed in Chapter 7.

The timeline is used to select the solve region, so that only samples in which cores are inside `KSPSolve` and none is inside `PCSetUp` contribute to the shares below. That approximates the repeated-solve phase from the profile alone. It is a time window, not the PETSc `RepeatedSolves` logging stage, and the two must not be quoted as the same quantity.

Table 5.2: Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. “OpenMP” is the GNU runtime’s spin and wait paths, that is threads with no work to do. Rows sum to under 100% because `libc`, the loader and the sampler make up a small remainder that is omitted. MAP instrumentation inflates these runs, so what the table establishes is the ordering between layouts; the absolute values are not transferable to the uninstrumented campaigns of Chapter 4.

Build	Nodes	Layout	PETSc	OpenMP	MPI	LibSci
LibSci	2	64×2	31.9%	36.5%	29.9%	1.0%
LibSci	2	32×4	51.6%	40.1%	5.8%	1.7%
LibSci	2	16×8	42.2%	53.5%	1.4%	2.1%
LibSci	8	64×2	4.5%	48.3%	46.7%	0.2%
LibSci	8	32×4	7.5%	69.3%	22.6%	0.3%
LibSci	8	16×8	15.6%	76.5%	6.2%	1.0%
Baseline	2	32×4	51.7%	40.8%	6.7%	0.0%
Baseline	2	16×8	40.2%	57.8%	1.4%	0.0%
Baseline	8	32×4	8.8%	68.6%	22.2%	0.0%
Baseline	8	16×8	16.0%	77.5%	6.0%	0.0%
Baseline	16	32×4	1.2%	69.8%	29.0%	0.0%

The two costs trade against each other. The MPI share falls as threads replace ranks: at eight nodes it goes 46.7%, 22.6%, 6.2% across 64×2 , 32×4 and 16×8 . That is the ordering Section 5.4 obtains from `-log_view` on the whole formal matrix, reproduced here by a different instrument on a different quantity. The time spent in the OpenMP runtime runs the other way, 48.3%, 69.3%, 76.5%, and it is not useful work: the frames are `cpu_relax`, `do_spin` and `__futex_wait`, that is threads waiting for something to do. The two builds agree wherever they overlap: across the four common cells the MPI shares differ by at most 0.9 percentage points and the OpenMP shares by at most 4.3, with no systematic sign.

Three limits apply to these numbers, and none can be removed from the profiles themselves. First, there is one profile per cell, so no observed range accompanies any percentage. Second, MAP is a sampling profiler designed for low overhead, but the overhead measured on these runs is neither small nor uniform: +35% on the timed stage and roughly threefold on `VecScatterEnd`, comparing the same configuration run with and without the profiler. Third, MAP's OpenMP instrumentation wraps the barrier and worksharing constructs whose behaviour is being measured, so it may influence that behaviour rather than only observe it. The vendor documents the sampling design but makes no quantitative claim about perturbation of OpenMP regions [10]. Instrumenting a barrier can only add work inside it, so the OpenMP shares below more likely overstate than understate an uninstrumented run. The claim made from this table is therefore the ordering between layouts, not the absolute percentage.

Within those limits, the table answers the question Chapter 4 left open, at the level of accounting rather than of mechanism. The question was what hybridisation costs, such that the cost has to be repaid before it helps. The cost is paid as idle time in the OpenMP runtime. The term that grows with threads per rank is time in `cpu_relax`, `do_spin` and `__futex_wait`, which are threads waiting, not time in the compute kernels. NUMA placement is excluded on topology alone: a rank of eight threads crosses a shared-L3 boundary but still fits inside one 16-core NUMA region, so no layout in this study places a rank across two NUMA regions and the four cannot differ in that respect (Table 2.2).

Neither is the growth a placement effect, because it is present at fixed layout, where a rank's threads occupy the same cores against the same shared-L3 boundary at every node count: holding 16×8 and going from two nodes to eight raises the waiting share from 53.5% to 76.5% on the LibSci build and from 57.8% to 77.5% on the baseline, as the work per threaded region shrinks.

That is the discriminating observation. It reads as OpenMP overhead becoming the dominant cost as the work per threaded region shrinks: more threads to synchronise over less work each. The fixed-layout result follows, because node count changes the work per region without changing the placement. The other two features of the shape, monotonicity in the thread count and a maximum where 16×8 performs worst, are consistent with that reading and equally consistent with a cache one, because threads

per rank and cores per L3 vary together across the four layouts. The profiles do not settle why the threads wait. A rank spanning two CCXs could produce load imbalance that is collected at the same barrier and would appear in the same `do_spin` frames. Section 5.4.4 leaves that to the hardware-counter work of Chapter 7. Section 5.4 gives the other half of the trade, on the full matrix and without a profiler.

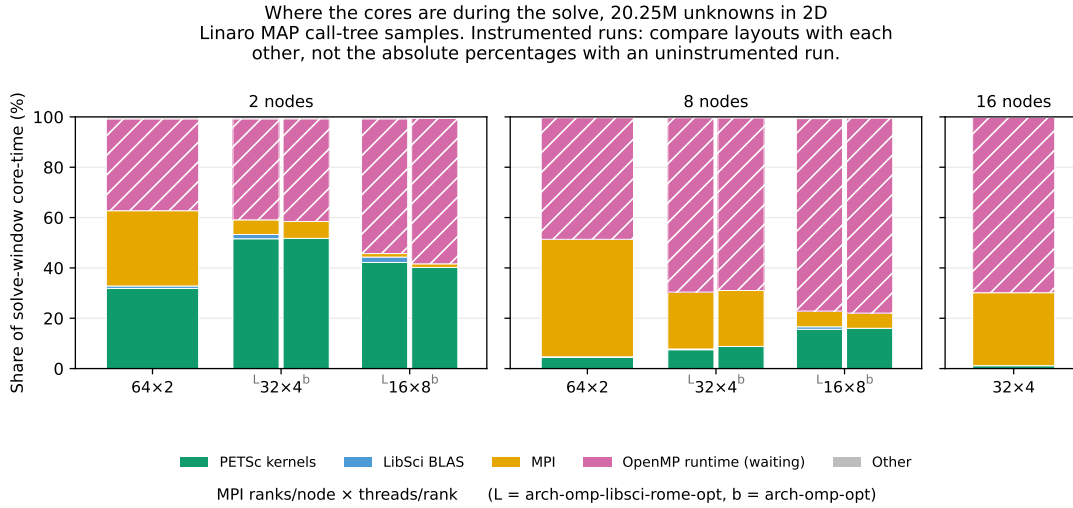


Figure 5.1: Where the cores are during the solve, from the MAP call trees.

A fourth observation is negative, and it closes the question Section 3.1 opened. Threaded LibSci is linked into the production build, and PETSc's sparse solve path barely calls it. The LibSci column of Table 5.2 never exceeds 2.1% of solve-window core-time in any of the six LibSci profiles, and it is 0.0% in every baseline profile, so the baseline build sampled no BLAS frame at all. This is the share of the sparse Krylov solve window, not of a whole application; a code dominated by dense factorisation or by matrix–matrix products would sit in a different regime. Here the threaded BLAS library cannot account for the layout ordering measured in Chapter 4. That is what Section 5.4.4 could otherwise only state as a limit of the evidence, and it is what allows the baseline and production builds to be treated as interchangeable in the four cells where both were profiled.

5.4 Bottleneck Identification

Section 5.3 leaves the hardware question open, but the `-log_view` event tables of the uninstrumented formal runs support a narrower question that Chapter 4 raised and could not answer: whether the layout effect tracks the share of the solve exposed to

inter-process synchronisation. All 264 formal runs are parsed here, so the evidence covers the same matrix as the scaling results and carries no profiler overhead.

5.4.1 What is Measured

For each run, the time of the halo-exchange events (`VecScatterBegin`, `VecScatterEnd`) and of CG's global-reduction events (`VecTDot`, `VecNorm`) is taken from the `RepeatedSolves` stage and divided by that stage's `KSPSolve` time. Call the result the *exposure share*.

Three limits apply to that quantity, and none is removable from `-log_view` alone. First, the synchronising events are logged *inside* `MatMult` and its relatives, so the exposure share is not a term in an additive time budget. Summing every non-container event against `KSPSolve` gives 116–372% across the formal runs, because `MatMult` is itself a container for the halo exchange: the same wait is counted once in `MatMult`, once in the `VecScatter` pair, and once again in `SFPack/SFUnpack`, which are excluded here for that reason.

Second, each event time that PETSc reports is a maximum over ranks, and different events may attain their maximum on different ranks, so the numerator sums maxima that need not have been attained on the rank that supplies the denominator. Third, `VecTDot` and `VecNorm` also perform local floating-point work, so their time is not purely communication.

The quantity is formed identically in every one of the 264 runs, so it remains comparable across layouts and node counts. It is an index of exposure, not an absolute time budget, and it is read only as such below.

5.4.2 Halo Exchange Dominates the Exposure, and Hybridisation Reduces It

Figure 5.2 and Figure 5.3 give the result. In both dimensions the share rises monotonically with node count for every layout, and at these size–node points it is strictly ordered by MPI rank count: fewer ranks, less exposure. The plotted share is the sum of the two event classes, which sit on disjoint call paths: the halo exchange is reached from `MatMult` and its relatives, the reductions directly from CG.

Table 5.3 splits the exposure along the $\sim 40k$ unknowns/core chain at four representative points in each dimension. Halo exchange is the larger of the two terms in every one of the 264 formal runs, by a factor of 1.7 to 2.9 in 2D and 2.9 to 5.7 in 3D. At 165m on 32 nodes flat MPI pays 34.3 percentage points to halo exchange in 2D and 70.9 in 3D, against 13.9 and 13.6 to the reductions; the totals these make up are the 48.2% and 84.5% at the right-hand end of Figure 5.2 and Figure 5.3. The 3D shares are far higher than the 2D shares at every point in the table, which is what the decomposition of Section 3.2 predicts: in 3D the $\pm mn$ coupling puts both k neighbours

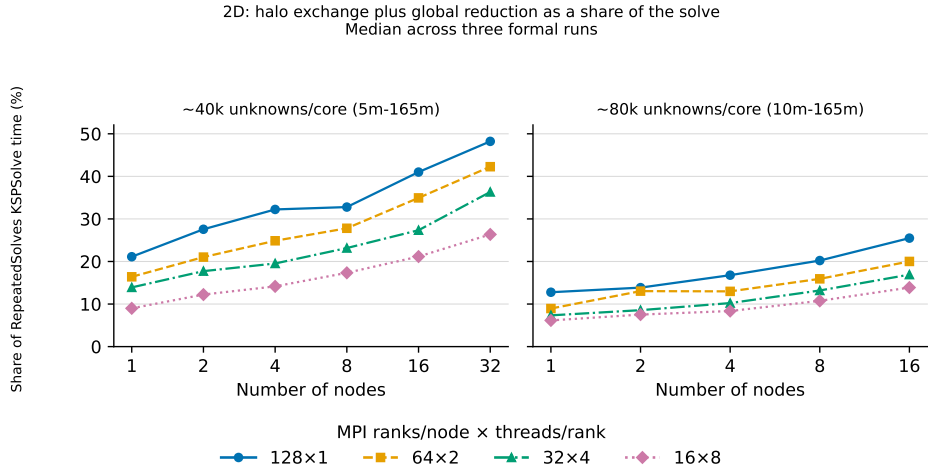


Figure 5.2: 2D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.

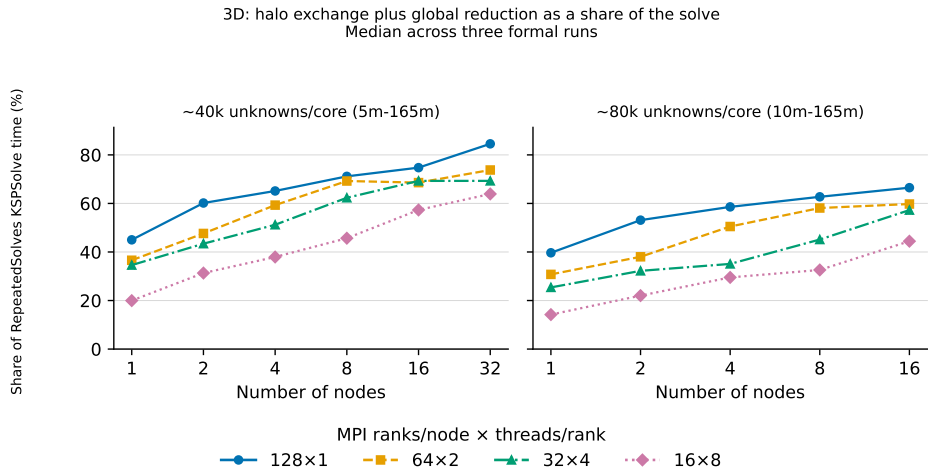


Figure 5.3: 3D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Each point is the median of three repetitions. Note the vertical scale: flat MPI reaches 84.6% at the largest point.

of every row off-rank once L drops below mn , while in 2D the matching $\pm m$ coupling never reaches more than $2m$ rows.

Table 5.3: The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is `VecScatterBegin` and `VecScatterEnd`; reduction is `VecTDot` and `VecNorm`. Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run; the two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of RepeatedSolves KSPSolve time.

Case	Nodes (size)	128×1		16×8	
		Halo	Reduction	Halo	Reduction
2D	1 (5m)	15.7	5.4	6.1	2.9
2D	8 (41m)	23.4	9.4	11.2	6.1
2D	16 (82m)	28.6	12.4	13.6	7.6
2D	32 (165m)	34.3	13.9	16.4	9.9
3D	1 (5m)	35.1	9.9	15.3	4.6
3D	8 (41m)	53.0	18.1	38.8	6.8
3D	16 (82m)	59.0	15.8	46.5	10.8
3D	32 (165m)	70.9	13.6	52.4	11.5

5.4.3 Reduced share Exposure

The ordering of table 5.3 alone cannot explain the results, because 16×8 has the lowest exposure at every point of Figure 5.2 and Figure 5.3 and, in 2D, the worst time per iteration at all 11 of them. What does track performance is the *size* of the reduction: comparing each layout's exposure against flat MPI at the same size–node point with its time per iteration relative to flat MPI gives Table 5.4.

Write $S_\ell(p)$ for the halo-plus-reduction share of layout ℓ at size–node point p , and $t_\ell(p)$ for its time per KSP iteration. The first two columns of Table 5.4 are the range,

$$\text{ExposureRemoved} = S_{128 \times 1} - S_\ell, \quad \text{TimeRatio} = \frac{t_\ell}{t_{128 \times 1}},$$

so $\text{ExposureRemoved} > 0$ means the layout is exposed less than flat MPI is, and $\text{TimeRatio} < 1$ means it is faster than flat MPI. For example, reading Table 5.3, 2D 16×8 at 165m on 32 nodes has $\text{ExposureRemoved} = (34.3 + 13.9) - (16.4 + 9.9) = 21.9$ points.

Break-even, a measure defined here, is computed by a and b :

$$\text{BreakEven} = \frac{1 - b}{a} = \frac{b - 1}{|a|}, \quad a < 0,$$

with no break-even defined when $a \geq 0$. Both a and b come from a least-squares fit over the 11 points of each layout:

$$\text{TimeRatio} = a \text{ExposureRemoved} + b$$

Here b is the intercept, the time ratio the line gives at $\text{ExposureRemoved} = 0$: what a layout would cost against flat MPI if it removed no exposure at all, so $b - 1$ is the cost of threading alone. The slope a is the change in TimeRatio per point of exposure removed, so $|a|$ is how much of that cost one point pays back.

Since $a < 0$, the fitted line drops below one exactly where ExposureRemoved passes BreakEven . The column is therefore a threshold: it is how much exposure a layout has to remove before threading pays for itself, and the lower it is, the sooner that happens. The fit is descriptive, so single points can fall on the wrong side of it.

The r column is the Pearson correlation, computed from the same pairs,

$$r = \frac{\text{cov}(\text{ExposureRemoved}, \text{TimeRatio})}{\sigma_{\text{ExposureRemoved}} \sigma_{\text{TimeRatio}}} \in [-1, 1],$$

their covariance over the product of their standard deviations. It shares the sign of a and says how closely they follow the line, so it measures how firm a break-even is rather than influencing its value.

Five of the six correlations r are negative and three exceed 0.8: the more exposure a layout removes at a given point, the faster it is relative to flat MPI. Read down b , the cost of threading before any exposure is removed: it rises sharply at each doubling, and at four and eight threads it is almost independent of the dimension, 1.1305 against 1.1270 and 1.4313 against 1.4226, although flat MPI's exposure share differs between 2D and 3D by 36 points at the largest size–node point. Only 64×2 , whose threading cost is small in both, differs appreciably. Read down $|a|$, the return on one point of removed exposure: it falls at every step, and at all three layouts it is larger in 3D than in 2D. The break-even column quantifies the trade-off: each doubling of threads per rank removes more exposure, but needs more of it removed before threading pays for itself.

Figure 5.4 and Figure 5.5 plot the fits. The 3D case is taken first because all three of its layouts behave the same way while the exception the argument turns on lies in 2D. In 3D the three layouts form three roughly parallel descending lines, displaced from each other along the horizontal axis: each removes more exposure than the one before it, and each needs more of it before crossing parity; the bands flatten as threads per rank rise, which is the a column.

Table 5.4: Reduction in synchronisation exposure against time per iteration, both relative to flat MPI at the same size–node point. The break-even value is the exposure reduction the fitted line requires to offset threading overhead. a and b are that line’s slope and intercept, so $(1 - b)/a$ reproduces the last column. Eleven points per layout; the fits are descriptive only.

Layout	Exposure removed	Time ratio	r	$10^3 a$	b	Break-even
2D						
64×2	0.8–7.4	0.872–0.995	−0.67	−15.29	1.0178	1.2
32×4	5.3–13.7	0.937–1.096	−0.59	−11.13	1.1305	11.7
16×8	6.3–21.9	1.294–1.674	+0.00	+0.08	1.4313	none
3D						
64×2	1.9–15.1	0.768–1.022	−0.86	−21.60	1.0618	2.9
32×4	5.5–23.5	0.751–1.070	−0.90	−17.47	1.1270	7.3
16×8	17.4–31.1	0.899–1.122	−0.83	−15.82	1.4226	26.7

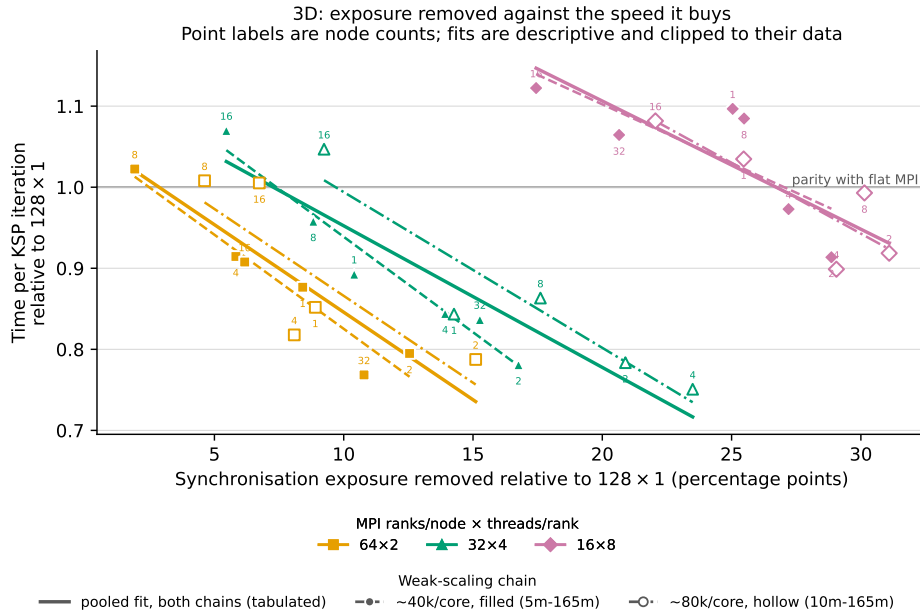


Figure 5.4: 3D ExposureRemoved relative to flat MPI against the resulting time per iteration, one point per size–node pair. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. Where a layout’s fitted line crosses parity is the break-even column of Table 5.4; the lines are descriptive, not predictive.

This explains two things.

First, why the 3D advantage rises and falls with node count:

Holding a layout and a chain fixed and walking the node count, the ExposureRemoved and the TimeRatio reverse together at every step. Along the $\sim 40k$ chain in 3D, 32×4 has an ExposureRemoved of 10.4, 16.8, 13.9, 8.8, 5.5 and 15.3 points from one node to 32, and a TimeRatio of 0.893, 0.781, 0.844, 0.958, 1.070 and 0.837: five transitions, five reversals, including the recovery at 32 nodes where a 9.8-point rise in ExposureRemoved turns a 7.0% loss into a 16.3% gain. All 18 transitions of 64×2 and 32×4 across both 3D chains behave this way. The advantage is non-monotone because ExposureRemoved is, not because the mechanism differs.

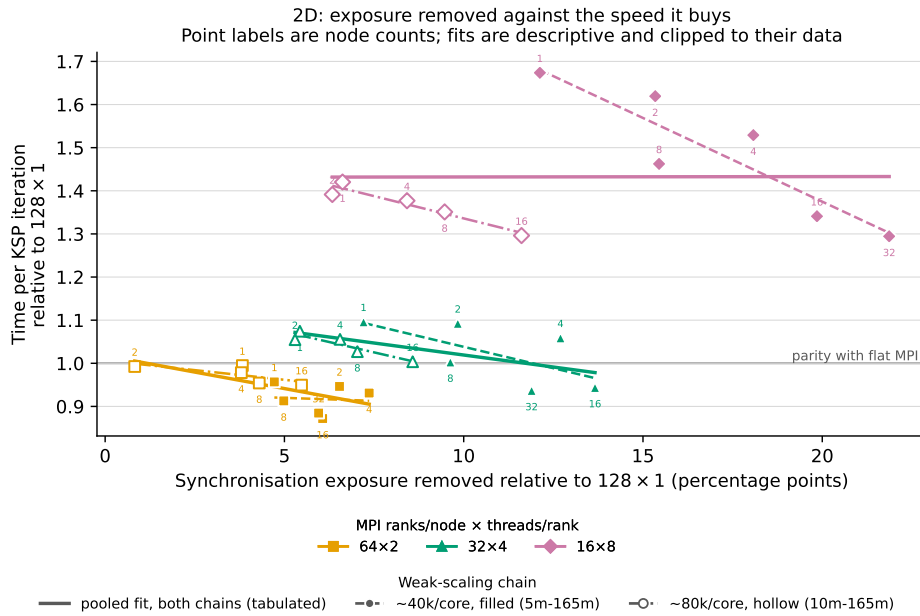


Figure 5.5: 2D ExposureRemoved relative to flat MPI against the resulting time per iteration, one point per size–node pair. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates; the two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. the lines are descriptive, not predictive.

The second is the threshold itself.

In 2D, 32×4 brackets its own 11.7-point break-even:

Along the $\sim 40k$ chain: an ExposureRemoved of 9.6 points at 41m on eight nodes for a TimeRatio of 1.003, then 13.7 points at 82m on 16 nodes for 0.944, crossing parity where the fit puts it. Along the $\sim 80k$ chain its ExposureRemoved never exceeds 8.6 points and it never crosses, bottoming at 1.004. The threshold holds in both directions.

But because the threshold is descriptive, it has a counter-example, and Figure 5.5 shows what it consists of: its pooled a is positive, so the solid line lies flat and the table returns no break-even at all (it is pointless), while the two broken lines from one per chain both descend steeply across it. The flatness is therefore an artefact of pooling. Taken separately the chains correlate at -0.91 over the six $\sim 40k$ points and -0.96 over the five $\sim 80k$ points, among the tightest fits in these experiments. On that reading, what the layout lacks is therefore not a relationship between `ExposureRemoved` and speed but the value of the `ExposureRemoved`, each chain's line reaches parity only beyond the largest `ExposureRemoved` the layout ever attains, 21.9 points, so neither threshold can be stated without extrapolating and neither is ever reached. That it is slower than flat MPI by 29–67% at every one of its 11 points is a direct observation and needs no fit. Some other cost dominates.

Table 5.2 identifies that cost: at the same problem size and node counts, 16×8 spends more of its core-time waiting in the OpenMP runtime than any other layout, 76.5% against 48.3% for 64×2 at eight nodes. Both halves of the trade are therefore measured, one on the full matrix without a profiler and the other on eleven profiled cells with one. What remains unmeasured is their sum: the profiled and unprofiled quantities come from different instruments on different runs and cannot be added into a single budget.

5.4.4 What This Does and Does Not Establish

Pooling all 33 hybrid measurements of a dimension breaks the relationship, and breaks it differently in each. In 2D the pooled correlation is $+0.58$, where the two layouts that have a break-even give -0.67 and -0.59 . In 3D it falls to $+0.10$ from three values that all exceed 0.8 in magnitude.

The obvious reason is that 16×8 removes the most exposure and is also the slowest. That is enough in 3D: drop it and the pooled figure returns to -0.75 . It is not enough in 2D, where dropping it still leaves $+0.13$. There, 32×4 removes more exposure than 64×2 at all 11 points and is slower at all 11. The ordering between layouts runs against the ordering within them in both dimensions, and 16×8 is only the clearest case. The relationship holds within a layout, so a pooled figure would mislead.

The two weak-scaling chains cannot be pooled either, and that is a second and independent effect: in 3D it does little harm, every layout's per-chain break-evens falling within 2.4 points of its pooled one, while in 2D none of the three layouts survives unchanged and 16×8 inverts the sign. Table 5.4 still reports pooled fits, because five and six points are too few to fit a chain on its own, and its columns should be read with that in mind. The b column agrees across dimensions only because the fits are pooled: split the chains and the agreement loosens at 32×4 and fails on the $\sim 40k$ chain at 16×8 . Neither pooling changes which layouts beat flat MPI, because that is read off the measurements and not off a fit.

These data show where the layout advantage comes from within PETSc's own event accounting: less exposure to halo exchange; global reduction is second and responds far less to the layout. Together with Section 5.3 they place the opposing cost in the OpenMP runtime rather than at a memory boundary. Neither shows a hardware mechanism. Several causes fit each half of the trade equally well, and neither instrument can tell them apart. Section 6.4 names the candidates; Chapter 7 gives the hardware-counter work that would separate them.

Chapter 6

Discussion

The four questions of Section 1.2 are answered in turn. RQ1 and RQ2 rest on Chapter 4, RQ3 on Chapter 5, and RQ4 on the limits of both.

6.1 RQ1: Does the threaded backend reduce time to solution, and by how much?

Hybrid execution cuts solve time in both dimensions. The two cases answer the question differently, and what differs is the shape of the result, not which layout wins

In 2D the answer is ordered. Along both unknowns/core chains, every hybrid layout gains on flat MPI as the node count grows, and four of the six series gain at every step. 64×2 is faster per iteration than flat MPI at all 11 size–node points, and its margin widens from 4.3% at one node to 12.8% at 16. 32×4 crosses from 9.6% slower at one node to 6.3% faster at 32. A layout comparison made at small node counts therefore does not carry over to large ones, which is the practical content of the 2D result.

In 3D the effect is larger but unordered. The best hybrid gain in throughput is 33.2%, against 14.7% in 2D, and two-thirds of the hybrid points are faster per iteration than flat MPI. No series gains at every step: the advantage peaks at two to four nodes, is gone by eight to 16, and returns at the largest point. On raw throughput 32×4 wins six of the 11 points, 64×2 four, and flat MPI one. The best balance depends on problem size and node count in a way the 2D data do not show.

6.2 RQ2: DAt which balance, and how does the answer change with the regime?

If it does, at which rank–thread balances, and how does the answer change with problem size, node count, dimensionality, and scaling regime?

Weak scaling

Strong scaling

and 16x8 and results

6.3 RQ3: Which parts of the solve account for the difference?

6.4 RQ4: How far do the trends generalise, and what would establish a mechanism?

How far do the observed trends generalise, and what would be needed to establish a mechanism for them?

Poisson problems

extend:

Chapter 7

Conclusion and Future Work

Appendix A

Supplementary Figures

A.1 Weak-scaling throughput

Figure A.1 and Figure A.2 give raw repeated-solve throughput for the weak-scaling campaign of Section 4.2. The dashed reference in each panel is ideal weak scaling, that is throughput proportional to the node count, anchored on the flat-MPI point at the panel's smallest node count; a layout that held its per-core rate exactly would run along it. Throughput rises almost linearly along both chains in both dimensions, and the gap that opens below the ideal line is the same loss that Table 4.4 reports as efficiency. The four layouts differ by only a few per cent against a range of three orders of magnitude, so these figures cannot carry the layout comparison. Figure 4.1 and Figure 4.3 do that instead.

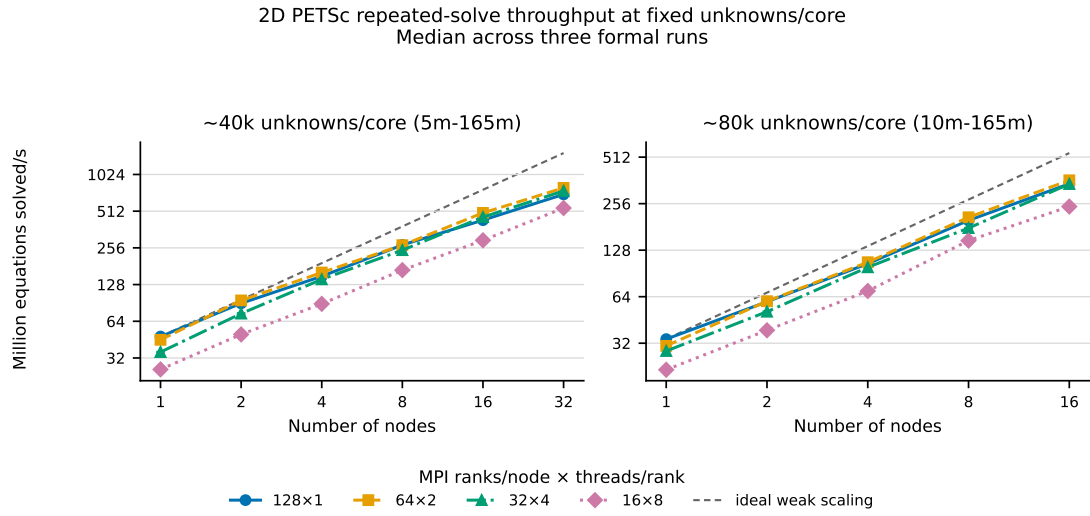


Figure A.1: Median 2D repeated-solve throughput along the two unknowns/core chains. The dashed grey line is ideal weak scaling.

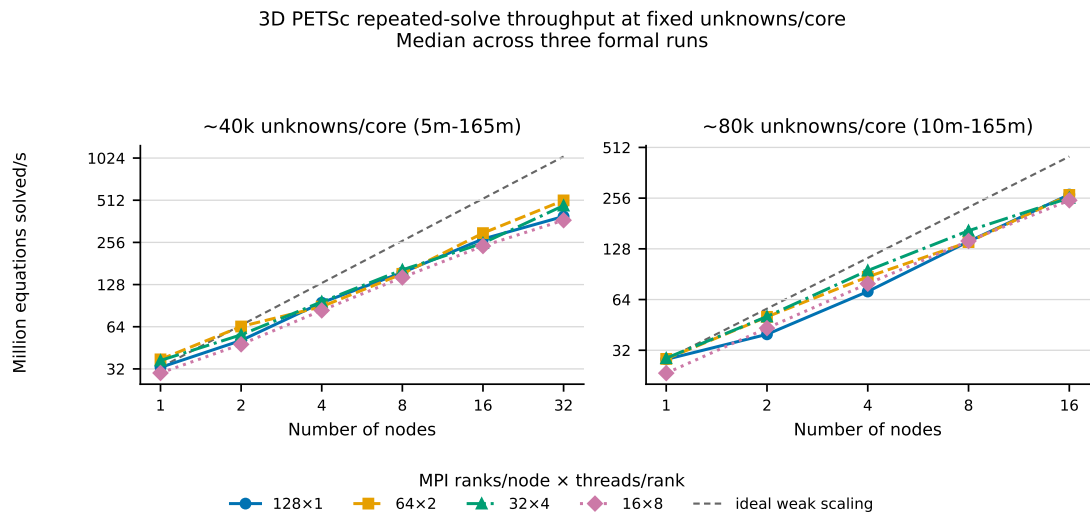


Figure A.2: Median 3D repeated-solve throughput along the two unknowns/core chains. The dashed grey line is ideal weak scaling.

A.2 Strong-scaling runtime grids

Section 4.1.2 presented these two campaigns as curves. Figure 4.5 showed the four 2D layouts falling together across the whole range and still falling at 32 nodes, and Figure 4.7 showed the 3D layouts separating instead, as three of the four reached a minimum and turned upward.

Table A.1 and Table A.2 give every point behind those two figures.

Table A.1: 2D strong-scaling campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.

Layout	Median s/solve at N nodes						16→32	Spread
	1	2	4	8	16	32		
128×1	0.7704	0.3399	0.1351	0.0558	0.0313	0.0282	1.11	7.9%
64×2	0.8260	0.3365	0.1265	0.0541	0.0348	0.0277	1.26	2.7%
32×4	0.8839	0.3944	0.1465	0.0632	0.0388	0.0317	1.22	10.8%
16×8	1.1325	0.5223	0.2272	0.0973	0.0568	0.0409	1.39	4.8%

Table A.2: 3D strong-scaling campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.

Layout	Median s/solve at N nodes						16→32	Spread
	1	2	4	8	16	32		
128×1	0.9066	0.5149	0.2208	0.0997	0.1776	0.2599	0.68	4.2%
64×2	0.8288	0.4041	0.2342	0.0936	0.0748	0.1703	0.44	3.2%
32×4	0.8022	0.4028	0.2145	0.1192	0.0652	0.0747	0.87	11.1%
16×8	0.9707	0.4718	0.2475	0.1330	0.1017	0.0681	1.49	7.6%

Appendix B

Stuff which is too detailed

Appendices should contain all the material which is considered too detailed to be included in the main body but which is, nevertheless, important enough to be included in the thesis.

Appendix C

Stuff which no-one will read

Some people include in their thesis a lot of detail, particularly computer code, which no-one will ever read. You should be careful that anything like this you include should contain some element of uniqueness which justifies its inclusion.

Bibliography

- [1] ARCHER2 Service. *ARCHER2 Hardware*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/hardware/> (visited on 07/22/2026).
- [2] ARCHER2 Service. *Performance Tuning*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/tuning/> (visited on 07/22/2026).
- [3] ARCHER2 Service. *ARCHER2: The UK National Supercomputing Service*. 2026. URL: <https://www.archer2.ac.uk/> (visited on 08/08/2026).
- [4] PETSc Development Team. *Changes: 3.22*. 2024. URL: <https://petsc.org/release/changes/322/> (visited on 07/22/2026).
- [5] PETSc Development Team. *KSP Tutorial ex2.c: Solves a Linear System in Parallel with KSP*. PETSc source tree, src/ksp/ksp/tutorials/ex2.c. 2025. URL: <https://petsc.org/release/src/ksp/ksp/tutorials/ex2.c.html> (visited on 08/08/2026).
- [6] Magnus R. Hestenes and Eduard Stiefel. “Methods of Conjugate Gradients for Solving Linear Systems”. In: *Journal of Research of the National Bureau of Standards* 49.6 (1952), pp. 409–436. DOI: 10.6028/jres.049.044.
- [7] Satish Balay et al. *PETSc/TAO Users Manual, Revision 3.24*. Tech. rep. ANL-21/39 Rev. 3.24. Argonne National Laboratory, 2025. DOI: 10.2172/2998643.
- [8] Satish Balay et al. “Efficient Management of Parallelism in Object-Oriented Numerical Software Libraries”. In: *Modern Software Tools in Scientific Computing*. Ed. by Eirik Arge, Are Magnus Bruaset, and Hans Petter Langtangen. Boston: Birkhäuser, 1997, pp. 163–202. DOI: 10.1007/978-1-4612-1986-6_8.
- [9] Klaus Stüben. “A Review of Algebraic Multigrid”. In: *Journal of Computational and Applied Mathematics* 128.1–2 (2001), pp. 281–309. DOI: 10.1016/S0377-0427(00)00516-1.
- [10] Linaro Limited. *Linaro MAP*. Linaro Forge 25.0.4 documentation. 2025. URL: https://docs.linaroforge.com/25.0.4/html/forge/forge/introduction_to_forge/map.html (visited on 07/22/2026).